Repaso: Medallero Olímpico

Este ejercicio se enmarca en el ámbito de los Juegos Olímpicos y su objetivo es la creación del medallero oficial (ranking de países). Para elaborar el ranking debes procesar los datos de las medallas (almacenados en un fichero en formato CSV), organizar esos datos en una colección y ordenarlos.

Usaremos datos de los Juegos Olímpicos de París 2024 que se encuentran en el fichero *medals.csv* (Kaggle, https://www.kaggle.com/datasets/piterfm/paris-2024-olympic-summer-games).

El criterio de ordenación para el medallero se basa en el número de medallas: primero oro, después plata y por último bronce. En caso de empate del número de medallas de los tres metales la ordenación se realiza alfabéticamente por el código de país.

Con este ejercicio queremos repasar contenidos trabajados en asignaturas previas que son interesante para abordar el contenido de la nueva asignatura que desarrollaremos durante este semestre:

- **Conceptos de orientación a objetos**: clases, atributos, constructores, enum, toString(), equals().
- **Lectura de ficheros de texto** (BufferedReader)
- **Collections**: List, Map, recorrido, acumulación de datos
- **Ordenación de colecciones**: Comparable (orden natural) y Comparator (órdenes alternativos).

Parte 1 - Sesión presencial en clase

El objetivo de esta primera parte es modelar las clases, leer el fichero y acumular los datos. Al final de la sesión el programa debe imprimir el medallero por consola (sin importar la ordenación).

Tarea 1 - Modelado de clases del dominio

Debes crear la siguientes clases:

- **TipoMedalla** (**enum**): ORO, PLATA, BRONCE.
- **Medalla**: representa una fila del CSV y contiene únicamente la información del país y el tipo de medalla. Crea el constructor, getters y toString().
- **Pais**: representa un país y tiene como atributos el código y el nombre del país. Crea el constructor, getters y toString().
- **MedalleroPais**: acumulador por país. Debe guardar el país y el número de medallas de oro, plata y bronce de dicho país. Valora la manera más adecuada para almacenar esta información. Crea también un método para obtener el número total de medallas de cada tipo de medalla.

Tarea 2 - Lectura del fichero

Crea la clase *MedalleroApp* que tendrá el método *main()*, leerá el fichero CSV e imprimirá por pantalla el medallero.

Para leer el contenido del fichero debes tener en cuenta lo siguiente:

- Procesa el fichero línea a línea utilizando la clase *BufferedReader* o Scanner.
- Lee la primera línea (cabecera) y localiza la posición que ocupan las 3 columnas que necesitamos para construir el medallero: *medal_type*, *country* y *country_code*. Usa el método *split()* para separar los datos de una línea.
- A partir de la segunda línea, extrae los datos, crea un objeto *Medalla.* Añade estos objetos a una *List<Medalla>*.
- Una vez procesado el fichero completo, imprime la lista de medallas. Formato libre, por ejemplo:

```
Listado de medallas:
Belgium - BEL -> GOLD
Italy - ITA -> SILVER
Belgium - BEL -> BRONCE
Australia - AUS -> GOLD
Great Britain - GBR -> SILVER
...
```

Tarea 3 - Acumulación en Collections

Recorre *List<Medalla>* y acumula los datos en un *Map<String, MedalleroPais>* (clave: nombre del país).

Tarea 4 - Salida provisional

Imprime los valores del Map (sin ordenar). Formato libre, por ejemplo:

```
Medallero sin ordenación:
DPR Korea -> GOLD: 0, SILVER: 2, BRONCE: 4, Total: 6
Kazakhstan -> GOLD: 1, SILVER: 3, BRONCE: 3, Total: 7
Portugal -> GOLD: 1, SILVER: 2, BRONCE: 1, Total: 4
Grenada -> GOLD: 0, SILVER: 0, BRONCE: 2, Total: 2
Greece -> GOLD: 1, SILVER: 1, BRONCE: 6, Total: 8
...
```

Parte 2 - Tarea para casa (≈2h)

El objetivo de esta parte es convertir el *Map<String, MedalleroPais>* en un ranking ordenado.

Tarea 5 - Orden natural con Comparable

Haz que la clase *MedalleroPais* implemente *Comparable<MedalleroPais>*. Implementa el método *compareTo()* con el criterio de ordenación oficial:

1. Número de medallas de oro primero.
2. Número de medallas de plata.
3. Número de medallas de bronce.
4. Orden alfabético por código de país.

Convierte *Map<String, MedalleroPais>* a una *List<PaisMedallero>*, ordena la lista con *Collections.sort()*.

Tarea 6 - Ranking ordenado

Imprime el ranking ordenado de países. Formato libre, por ejemplo:

```
Medallero con ordenación oficial:
1. United States -> GOLD: 40, SILVER: 44, BRONCE: 41, Total: 125
2. China -> GOLD: 40, SILVER: 27, BRONCE: 24, Total: 91
3. Japan -> GOLD: 20, SILVER: 12, BRONCE: 13, Total: 45
4. Australia -> GOLD: 18, SILVER: 19, BRONCE: 16, Total: 53
...
```

Tarea 6 - Ordenaciones alternativas con Comparator

Sin modificar *MedalleroPais*, crea dos comparadores adicionales *Comparator<MedalleroPais>*:

- Por número total de medallas (sin importar el metal). Ejemplo de impresión:

```
Medallero con ordenación por total de medallas:
1. United States -> GOLD: 40, SILVER: 44, BRONCE: 41, Total: 125
2. China -> GOLD: 40, SILVER: 27, BRONCE: 24, Total: 91
3. Great Britain -> GOLD: 14, SILVER: 22, BRONCE: 29, Total: 65
4. France -> GOLD: 16, SILVER: 26, BRONCE: 22, Total: 64
...
```

- Por orden alfabético del nombre del país. Ejemplo de impresión:

```
Medallero con ordenación por nombre de país:
1. "Hong Kong -> GOLD: 2, SILVER: 0, BRONCE: 2, Total: 4
2. AIN -> GOLD: 1, SILVER: 3, BRONCE: 1, Total: 5
3. Albania -> GOLD: 0, SILVER: 0, BRONCE: 2, Total: 2
4. Algeria -> GOLD: 2, SILVER: 0, BRONCE: 1, Total: 3
...
```

Opcional

- Imprimir el ranking numerado usando el mismo número si 2 países empatan en la comparación de número de medallas (obviando el código del país). Ejemplo de impresión:

```
...
55. Armenia -> GOLD: 0, SILVER: 3, BRONCE: 1, Total: 4
55. Colombia -> GOLD: 0, SILVER: 3, BRONCE: 1, Total: 4
56. DPR Korea -> GOLD: 0, SILVER: 2, BRONCE: 4, Total: 6
56. Kyrgyzstan -> GOLD: 0, SILVER: 2, BRONCE: 4, Total: 6
...
```

- Exportar el medallero ordenado a un fichero CSV. Ejemplo del formato del fichero:

```
country_code,country,gold_medals,silver_medals,bronze_medals,total_medals
USA,United States,40,44,41,125
CHN,China,40,27,24,91
JPN,Japan,20,12,13,45
AUS,Australia,18,19,16,53
...
```